

Replay Gluing: Deterministic Reconstruction of Global Sections from Trace Logs

Halley Young
Microsoft Research

Abstract

Formal verification is only as trustworthy as the pipeline that produces it. When a verification run is interrupted, audited after the fact, or migrated to a new environment, users need a guarantee that re-running the system from recorded descent steps yields *exactly* the same global section as the original run. We present *Replay Gluing*, a framework implemented in JUGE_O's `generation/replay_gluing/` sub-package that reconstructs global sections deterministically from trace logs. The `ReplayGluer` class ingests a structured log of GLUE descent steps, applies one of four `GluingStrategy` policies (full, incremental, lazy, adaptive), and produces a `GlobalSection` whose hash is provably identical to the original under deterministic program execution. Partial replay from checkpoints and semantic-closure completion via `SemanticClosure` handle the practically important case where logs are truncated. A divergence detector flags any discrepancy between replayed and cached results, turning silent corruption into a loud, auditable error. We prove the *Replay Determinism Theorem*: for any deterministic verification task, the replayed global section equals the original bit-for-bit. Experiments on 300 benchmarks show median replay overhead of 1.8ms over the original run, with incremental replay recovering 92% of unchanged local sections from cache.

Keywords

replay; descent; global section; gluing; determinism; auditability; checkpoints; semantic closure; trace log; formal verification

1 Introduction

Verification pipelines are long-lived artifacts. A proof of memory safety produced today may need to be re-checked six months from now against a patched dependency, audited by a regulator who did not witness the original run, or resumed after a hardware fault interrupted a forty-hour SMT session. Each of these scenarios demands the same property: *reproducibility*—the ability to reconstruct, from a compact log of past actions, the same proof artifact that was assembled originally.

In the sheaf-theoretic model underlying JUGE_O [1], a completed verification is a *global section* $\sigma \in \Sigma$: a coherent assignment of local proof data across all coordinates of the program. A global section is assembled through a series of GLUE descent steps that paste local sections over compatible overlaps. The trace log $\mathcal{L}$ records precisely these steps—which patches were glued in which order, which cache entries were consulted, and what overlap data was computed.

The central question of this paper is: given $\mathcal{L}$, can we reconstruct σ faithfully? The answer is *yes*, provided: (i) the underlying verification computations are deterministic; (ii) the gluing operations

respect the cocycle condition; and (iii) semantic closure is computed for any patches absent from a truncated log.

Contributions.

1. We define the **replay model** (§2): a formal map $\text{ReplayGluer}: \mathcal{L} \rightarrow \Sigma$ that reconstructs global sections from trace logs.
2. We catalogue four **gluing strategies** (§3) that trade completeness for efficiency: FULL, INCR, LAZY, and ADAPT.
3. We show how **semantic closure** (§4) fills gaps introduced by log truncation or checkpoint resumption.
4. We characterise **divergence** (§5) and prove that divergence is detectable whenever the original hash is stored.
5. We describe **cache integration** (§6): how replay reuses intermediate results to avoid redundant SMT calls.
6. We prove the **Replay Determinism Theorem** (§7): for deterministic programs, replay is bit-for-bit identical to the original.
7. We validate the system experimentally (§8) on 300 real verification tasks.

Related work. Proof replaying is well-studied in interactive theorem provers: COQ’s **Proof General** checkpointing [4], LEAN 4’s incremental elaboration [5], and DAFNY’s caching layer [6]. Unlike those systems, JUGEO’s replay operates at the *sheaf* level: we are not replaying tactic scripts but reconstructing the global section of a verification sheaf from descent-step logs. Deterministic replay for distributed systems [7] is a closer analogy, but concerns process schedules rather than mathematical coherence conditions.

2 Replay Model

2.1 Semantic site and global sections

We recap the sheaf-theoretic setting from Papers 01 and 03. Let $(\mathcal{S}, \text{Cov})$ be the JUGEO semantic site whose objects are program coordinates $c \in \text{Ob}(\mathcal{S})$. A *pre-sheaf of judgments* $\mathcal{F}: \mathcal{S}^{\text{op}} \rightarrow \mathbf{Set}$ assigns to each coordinate c a set $\mathcal{F}(c)$ of candidate judgments. Restriction maps $\text{res}_{c \rightarrow d}: \mathcal{F}(c) \rightarrow \mathcal{F}(d)$ encode how evidence propagates downward.

A *local section* over a cover $\{U_i \rightarrow U\}_{i \in I}$ is a family $(\sigma_i)_{i \in I}$ with $\sigma_i \in \mathcal{F}(U_i)$. A *global section* is a local section that satisfies the gluing axiom: for all pairs i, j ,

$$\text{res}_{U_i \cap U_j \leftarrow U_i}(\sigma_i) = \text{res}_{U_i \cap U_j \leftarrow U_j}(\sigma_j).$$

The GLUE descent step (Paper 03, Definition 3.4) assembles a global section σ from compatible local sections by iterating over all overlap pairs.

2.2 Trace logs

Definition 2.1 (Trace log). A *trace log* $\mathcal{L}$ is a finite sequence of *descent records*

$$\mathcal{L} = \langle r_1, r_2, \dots, r_n \rangle,$$

where each record $r_k = (\mathbf{p}_k, \sigma_k, \mathbf{D}_k, t_k)$ encodes the patch identifier $\mathbf{p}_k \in \mathcal{P}$, the local section σ_k produced, the gluing data $\mathbf{D}_k$ (overlap maps and cocycle witnesses), and a monotone timestamp t_k .

Trace logs are emitted by JUGEO’s descent engine whenever the `TraceMode` flag is enabled. They are stored as append-only NDJSON files; each line is one descent record serialised to JSON.

2.3 The ReplayGluer

Definition 2.2 (ReplayGluer). The *replay gluer* is the map

$$\text{ReplayGluer}: \mathcal{L} \times \text{GluingStrategy} \longrightarrow \Sigma$$

defined by the following algorithm:

1. Parse $\mathcal{L}$ into a *gluing plan* $\text{Plan} = (\mathcal{P}, \Delta, \bar{\Delta}, \mathcal{G}, \text{GluingStrategy})$, where $\mathcal{P} = \Delta \sqcup \bar{\Delta}$.
2. Order patches according to `GluingStrategy` into a schedule $\pi: \{1, \dots, |\mathcal{P}|\} \rightarrow \mathcal{P}$.
3. Execute the gluing loop: for $k = 1$ to $|\mathcal{P}|$, set $\sigma_{\pi(k)} \leftarrow \text{replay}(\pi(k), \text{Plan})$, verifying that all overlap conditions with previously glued patches are satisfied.
4. Return $\sigma = \text{glue}(\{\sigma_p\}_{p \in \mathcal{P}})$.

The function `replay(p, Plan)` inspects the cache (via κ) and re-executes the original descent step only if the cache entry is stale or absent. Algorithm ?? gives the complete pseudocode.

Listing 1: Core replay loop in `ReplayGluer.run()`.

```

1 class ReplayGluer:
2     def run(self,
3         log: TraceLog,
4         strategy: GluingStrategy,
5         cache: SectionCache | None = None) -> GlobalSection:
6         plan = self._build_plan(log, strategy)
7         algo = select_algorithm(plan.strategy)
8         errors = algo.pre_check(plan)
9         if errors:
10             raise ReplayError(errors)
11         gluing = algo.execute(plan)           # fills patch_sections
12         gluing = self._fill_semantic_closure(gluing, cache)
13         diverged = self._detect_divergence(gluing, log)
14         if diverged:
15             raise DivergenceError(diverged)
16         return self._assemble_global_section(gluing)

```

Definition 2.3 (Replay plan). A *replay plan* $\text{Plan} = (\mathcal{P}, \Delta, \bar{\Delta}, \mathcal{G}, s)$ consists of:

- $\mathcal{P}$: the full set of patches appearing in $\mathcal{L}$;
- $\Delta \subseteq \mathcal{P}$: patches whose cached section differs from the logged section (detected by hash comparison);
- $\bar{\Delta} = \mathcal{P} \setminus \Delta$: patches whose cache is still valid;
- $\mathcal{G}: \mathcal{P} \rightarrow 2^{\mathcal{P}}$: upstream dependency graph derived from overlap structure;
- $s \in \{\text{FULL}, \text{INCR}, \text{LAZY}, \text{ADAPT}\}$: the chosen gluing strategy.

Proposition 2.4 (Plan validity). *A replay plan is valid if and only if $\Delta \cap \bar{\Delta} = \emptyset$ and $\mathcal{G}$ is acyclic. The `JUGEO.is_valid()` method checks both conditions in $O(|\mathcal{P}|^2)$.*

3 Gluing Strategies

The `GluingStrategy` enumeration governs which patches are re-executed from scratch and which are served from cache. The four strategies form a spectrum from maximal trust to minimal trust.

3.1 Full replay (FULL)

Under FULL, every patch $\mathbf{p} \in \mathcal{P}$ is re-executed regardless of cache status. This provides the strongest guarantees but is the most expensive.

Definition 3.1 (Full replay). Let $\mathcal{F}_{\text{orig}}$ be the verification sheaf from the original run. A *full replay* under strategy FULL produces $\text{ReplayGluer}(\mathcal{L}, \text{FULL}) = \sigma$ such that for all $\mathbf{p} \in \mathcal{P}$,

$$\sigma|_{\mathbf{p}} = \text{fresh_descent}(\mathbf{p}),$$

i.e. no cache entries are consumed.

Full replay is appropriate when (a) the trust policy requires an independent witness, or (b) the cache is suspected to be corrupted.

3.2 Incremental replay (INCR)

Incremental replay is the default strategy in JUGEO. Only patches in Δ (and their dependants in $\mathcal{G}$) are re-executed; the rest are served from cache.

Definition 3.2 (Blast radius). Given dependency graph $\mathcal{G}$, the *blast radius* of a changed set Δ is

$$\beta(\Delta, \mathcal{G}) = \{\mathbf{p} \in \mathcal{P} \mid \exists q \in \Delta \text{ s.t. } q \in \text{ancestors}_{\mathcal{G}}(\mathbf{p})\}.$$

Incremental replay re-executes exactly $\beta(\Delta, \mathcal{G})$.

Proposition 3.3 (Soundness of incremental replay). *If $\text{cached_hash}(\mathbf{p}) = \text{log_hash}(\mathbf{p})$ for all $\mathbf{p} \in \bar{\Delta}$, then incremental replay produces the same global section as full replay:*

$$\text{ReplayGluer}(\mathcal{L}, \text{INCR}) = \text{ReplayGluer}(\mathcal{L}, \text{FULL}).$$

Proof. By induction on the topological order of $\mathcal{G}$. Base case: patches with no predecessors satisfy the hash condition by hypothesis. Inductive step: a patch $\mathbf{p} \notin \beta(\Delta, \mathcal{G})$ has all predecessors in $\bar{\Delta}$ with valid caches; by the descent axiom the gluing data is identical to the original run, hence $\sigma|_{\mathbf{p}}$ is identical. $\square$

3.3 Lazy replay (LAZY)

Under LAZY, patches in $\bar{\Delta}$ are *deferred*: they are neither re-executed nor loaded from cache unless downstream patches request their overlap data. This minimises I/O in auditing scenarios where only a subset of the global section is inspected.

Definition 3.4 (Deferred patch set). The *deferred set* of a lazy replay is $\mathcal{D} = \bar{\Delta} \setminus \text{reachable}(\Delta, \mathcal{G}^{\text{op}})$, where $\mathcal{G}^{\text{op}}$ is the reverse dependency graph.

3.4 Adaptive replay (ADAPT)

Adaptive replay estimates the cost of each patch using the stored `ReplayMetrics` and dynamically switches between INCR and LAZY to minimise total replay time subject to a time budget T_{max} .

Definition 3.5 (Adaptive policy). Under ADAPT, the scheduler assigns each patch $\mathbf{p}$ to the *lazy tier* if $\text{cost}(\mathbf{p}) > T_{\text{max}}/|\mathcal{P}|$ and the *incremental tier* otherwise. The cost estimate $\text{cost}(\mathbf{p})$ is derived from the `estimated_ms` field in the trace log.

3.5 Strategy comparison

Table 1 summarises the four strategies along four dimensions: patches re-executed, cache reads, deferred patches, and worst-case time complexity relative to a full replay of n patches with maximum dependency depth d .

Table 1: Comparison of gluing strategies.

Strategy	Re-executed	Cache reads	Deferred	Time bound
FULL	$ \mathcal{P} $	0	0	$O(n \cdot T_{\mathbf{p}})$
INCR	$ \beta(\Delta) $	$ \bar{\Delta} $	0	$O((\Delta + d) \cdot T_{\mathbf{p}})$
LAZY	$ \Delta $	partial	$ \mathcal{D} $	$O(\Delta \cdot T_{\mathbf{p}})$
ADAPT	dynamic	dynamic	dynamic	$\leq T_{\text{max}} + O(n)$

4 Semantic Closure of Partial Replays

When a trace log is truncated—because the original run was interrupted before completion, or because only a prefix was archived—the replay produces a *partial global section*: a coherent family of local sections over a proper sub-cover $\{V_i\}_i \subsetneq \{U_i\}_i$. Before the partial section can be used for downstream queries, it must be *closed* under the sheaf axioms.

4.1 SemanticClosure

The `SemanticClosure` class in `generation/semantic_closure/` implements closure completion.

Definition 4.1 (Semantic closure). Let σ^{part} be a partial global section defined over $\mathcal{P}^{\text{part}} \subseteq \mathcal{P}$. The *semantic closure* $\bar{\mathcal{F}}(\sigma^{\text{part}})$ is the smallest extension to a global section over all of $\mathcal{P}$ that:

1. agrees with σ^{part} on $\mathcal{P}^{\text{part}}$;
2. satisfies the cocycle condition on all overlaps; and

3. assigns the lowest available trust tier UNVERIFIED to any $p \in \mathcal{P} \setminus \mathcal{P}^{\text{part}}$ for which no cached section exists.

Remark 4.2. The closure is not unique if multiple consistent extensions exist. JUGE0 breaks ties by choosing the extension with the highest total trust mass $\sum_p \mathcal{T}(\sigma|_p)$, computed greedily in topological order.

4.2 Closure algorithm

Algorithm 2 shows the closure computation:

Listing 2: Semantic closure completion in `SemanticClosure.complete()`.

```

1 class SemanticClosure:
2     def complete(self,
3         partial: PartialGlobalSection,
4         cache: SectionCache) -> GlobalSection:
5         pending = partial.missing_patches()
6         for patch in topological_order(pending, partial.dep_graph):
7             cached = cache.lookup(patch)
8             if cached is not None and cached.trust >= UNVERIFIED:
9                 partial.add(patch, cached)
10            else:
11                # synthesise minimal section: obligations unknown
12                partial.add(patch, MinimalSection(patch,
13                    trust=UNVERIFIED))
14                # enforce cocycle on all overlaps with already-present
15                # neighbours
16                for nbr in partial.present_neighbours(patch):
17                    overlap_key = (patch, nbr)
18                    partial.enforce_cocycle(overlap_key)
19        return partial.to_global_section()

```

Theorem 4.3 (Closure termination and coherence). *For any partial global section σ^{part} with acyclic dependency graph, Algorithm 2 terminates in $O(|\mathcal{P}|^2)$ time and produces a coherent global section.*

Proof. Termination follows from the strict decrease of $|\mathcal{P} \setminus \mathcal{P}^{\text{part}}|$ at each iteration. Coherence: the loop maintains the invariant that all patches added so far satisfy the cocycle condition with their present neighbours; the `enforce_cocycle` step extends this invariant to the newly added patch. $\square$

4.3 Checkpoint-based partial replay

Checkpoints are a special case of partial replay. A *checkpoint* ckpt_k is a snapshot of the gluing state after processing the first k patches in schedule order.

Definition 4.4 (Checkpoint-based replay). Given checkpoint ckpt_k and suffix $\log \mathcal{L}[k+1 : n]$, replay *resumes* by:

1. Deserialising ckpt_k into a partial global section σ_k^{part} ;
2. Applying the remaining descent records from $\mathcal{L}[k+1 : n]$;

3. Applying semantic closure if any patches in $\mathcal{P} \setminus \mathcal{P}_k^{\text{part}}$ are not covered by $\mathcal{L}[k + 1 : n]$.

Corollary 4.5. *Checkpoint-based replay is sound: it produces a global section identical to one that processes the full log $\mathcal{L}[1 : n]$, provided ckpt_k is a valid snapshot of the gluing state at step k .*

5 Divergence Detection

5.1 Sources of divergence

Replay can diverge from the original run for several reasons:

1. **Non-determinism:** the verification oracle (SMT solver, random tester) produces different output on re-execution.
2. **Environmental change:** a tool version upgrade alters solver behaviour.
3. **Log corruption:** the trace log was partially overwritten.
4. **Stale cache:** a cache entry was silently invalidated (e.g., by a file-system rename).

Cases (1)–(2) are *legitimate* divergences that indicate a real change in verification status; cases (3)–(4) are *artefact* divergences that indicate infrastructure failure.

5.2 Divergence set

Definition 5.1 (Divergence set). Given original trace log $\mathcal{L}$ and replay result σ^{REPLAY} , the *divergence set* is

$$\text{Div}(\mathcal{L}, \sigma^{\text{REPLAY}}) = \{\mathbf{p} \in \mathcal{P} \mid \text{hash}(\sigma^{\text{REPLAY}}|_{\mathbf{p}}) \neq r_{\mathbf{p}}.\text{hash}\},$$

where $r_{\mathbf{p}} \in \mathcal{L}$ is the descent record for $\mathbf{p}$ and $r_{\mathbf{p}}.\text{hash}$ is the stored SHA-256 hash of the original local section.

Proposition 5.2 (Divergence detectability). *Divergence is detectable if and only if the original trace log contains a `hash` field for each descent record. If $\text{Div}(\mathcal{L}, \sigma^{\text{REPLAY}}) \neq \emptyset$, the `ReplayGluer` raises `DivergenceError` with a structured report identifying each diverged patch and the expected vs. observed hash.*

5.3 Divergence classification

The `DivergenceError` report classifies each diverged patch as:

- **Structural:** the patch’s local section type has changed (e.g. a previously scalar section became a set-valued one).
- **Value:** the type is the same but the computed value differs.
- **Trust:** only the trust tier has changed (value is identical modulo trust annotation).

Trust-only divergences are promoted through JU GEO’s PROMOTE/CHALLENGE pipeline rather than treated as errors, since they may reflect legitimate evidence accumulation between runs.

Listing 3: Divergence detection and classification.

```

1 def _detect_divergence(
2     self,
3     gluing: GluingUnderReplay,
4     log: TraceLog) -> list[DivergenceRecord]:
5     results = []
6     for patch, section in gluing.patch_sections.items():
7         record = log.get_record(patch)
8         if record is None or record.hash is None:
9             continue
10        observed = sha256(section)
11        if observed != record.hash:
12            results.append(DivergenceRecord(
13                patch=patch,
14                expected=record.hash,
15                observed=observed,
16                kind=classify_divergence(record.section, section),
17            ))
18    return results

```

6 Integration with Caching

JUGEO’s section cache κ is a content-addressed store: each entry is keyed by $(p, \text{hash}(\text{inputs}))$ and stores the computed local section together with provenance metadata.

6.1 Cache coherence during replay

During replay the cache serves as a secondary oracle: the `replay` function queries $\kappa(p)$ before re-executing the descent step. Cache coherence is maintained by the following protocol.

Definition 6.1 (Cache coherence). The cache κ is *replay-coherent* with trace log $\mathcal{L}$ if, for every $p \in \bar{\Delta}$,

$$\kappa(p) = r_p.\text{section} \text{ and } \text{hash}(\kappa(p)) = r_p.\text{hash}.$$

Lemma 6.2 (Cache eviction safety). *If κ is replay-coherent and the incremental replay algorithm evicts entry $\kappa(p)$ for some $p \in \bar{\Delta}$, then the replayed section $\sigma^{\text{REPLAY}}|_p$ is still correct, provided the eviction triggers a fresh descent step with the same input hash.*

Proof. By determinism of the descent step: if the input to the descent computation for p is identical to the original (guaranteed by the unchanged input hash), the output is identical. The eviction therefore causes a cache miss that resolves to the correct value. $\square$

6.2 Two-level cache architecture

In practice JUGEO uses a two-level cache: a fast in-memory LRU cache for hot patches and a persistent disk cache (SQLite) for the full history. Replay first queries the in-memory tier, then falls through to disk. The disk tier is always authoritative; the in-memory tier is a performance optimisation.

Listing 4: Two-level cache lookup during replay.

```

1 class TwoLevelSectionCache:
2     def lookup(self, patch: str) -> LocalSection | None:
3         # L1: in-memory LRU
4         if (entry := self._l1.get(patch)) is not None:
5             return entry
6         # L2: persistent SQLite store
7         if (entry := self._l2.fetch(patch)) is not None:
8             self._l1[patch] = entry # promote to L1
9             return entry
10        return None

```

6.3 Replay metrics and cache statistics

Each `ReplayGluingPlan` carries a `ReplayMetrics` object that records, after execution: total duration in milliseconds, the estimated cost (in abstract units), the number of patches touched, the L1 and L2 cache hit rates, and the number of diverged patches. These metrics are written to the audit trail alongside the reconstructed global section.

7 The Replay Determinism Theorem

7.1 Deterministic verification tasks

Definition 7.1 (Deterministic task). A verification task T is *deterministic* if, for every patch $p \in \mathcal{P}$, the descent function $\text{desc}: \text{Inputs}(p) \rightarrow \text{Section}(p)$ is a pure function: given the same inputs, it always produces the same output. We write $T \in \text{Det}$ to indicate this.

Determinism holds for tasks whose oracles are:

- SMT solvers run with a fixed seed and time limit;
- LEAN 4 and COQ elaborators (definitionally deterministic);
- Static analysers with no randomisation.

7.2 Statement and proof

Theorem 7.2 (Replay Determinism). *Let $T \in \text{Det}$ be a deterministic verification task, let $\mathcal{L}$ be a complete trace log from a successful run of T , and let σ^{orig} be the global section produced by that run. Then for any strategy $s \in \{\text{FULL}, \text{INCR}, \text{LAZY}, \text{ADAPT}\}$,*

$$\text{ReplayGluer}(\mathcal{L}, s) = \sigma^{\text{orig}},$$

where equality is bit-for-bit equality of the serialised global section.

Proof. We proceed by structural induction on the topological order of $\mathcal{G}$.

Base case. Let p_0 be a patch with no predecessors in $\mathcal{G}$. Its inputs are determined entirely by the task specification T (program text and annotation), which is unchanged between original run and replay. By determinism ($T \in \text{Det}$), $\text{desc}(\text{Inputs}(p_0))$ produces the same local section in both runs. Whether $p_0 \in \Delta$ or $\bar{\Delta}$, the replayed section equals the original:

- If $p_0 \in \Delta$: the descent step is re-executed with identical inputs, yielding the same output by determinism.
- If $p_0 \in \bar{\Delta}$: the cache returns the stored value, which equals the original by cache coherence (Definition 6.1).

Inductive step. Assume all predecessors $\text{pred}_{\mathcal{G}}(p)$ have been replayed correctly. Their replayed sections form the inputs to p 's descent step. By the inductive hypothesis those inputs are identical to the original run. By determinism the descent step produces the same output.

Global assembly. By induction, every patch's local section is bit-for-bit equal to the original. The cocycle conditions are therefore also satisfied with the same witness data. The GLUE assembly step (a deterministic function of the local sections and overlap maps) therefore yields the same global section.

Strategy-independence. For each of the four strategies, re-executed patches use deterministic descent (as above) and cached patches use coherent cache entries (Lemma 6.2). Deferred patches under LAZY are either requested by a downstream query (and then replayed correctly by the inductive argument) or never accessed (and thus immaterial to the output). $\square$

Corollary 7.3 (Hash stability). *Under the conditions of Theorem 7.2, $\text{hash}(\text{ReplayGluer}(\mathcal{L}, s)) = \text{hash}(\sigma^{\text{orig}})$, so the divergence set $\text{Div}(\mathcal{L}, \text{ReplayGluer}(\mathcal{L}, s)) = \emptyset$.*

Remark 7.4. The theorem does *not* hold for non-deterministic tasks ($T \notin \text{Det}$). For example, a task relying on randomised testing produces different counter-examples on each run. In that case replay can still be useful—it reconstructs one valid global section—but bit-for-bit identity with the original is not guaranteed. The divergence detection mechanism (§5) surfaces these differences explicitly.

7.3 Auxiliary lemmas

The following lemmas underpin the proof and are of independent interest.

Lemma 7.5 (Overlap preservation). *If local sections σ_i and σ_j satisfy the cocycle condition in the original run, and both are replayed identically (by Theorem 7.2), then the replayed sections σ_i^{REPLAY} and σ_j^{REPLAY} also satisfy the cocycle condition on $U_i \cap U_j$.*

Proof. By identity of the sections and the determinism of restriction maps. $\square$

Lemma 7.6 (Log completeness). *A trace log $\mathcal{L}$ is complete for T if every descent step of T appears as a record in $\mathcal{L}$. For complete logs, the closure step in Algorithm 2 is a no-op: no patches need to be synthesised.*

Proof. Immediate from Definition 4.1: if $\mathcal{P}^{\text{part}} = \mathcal{P}$ then no closure completion is needed. $\square$

8 Experiments

8.1 Setup

We evaluate replay gluing on three benchmark suites drawn from the standard JUGE evaluation corpus:

Standard (300 tasks)

The same 300 real Python programs from Paper 10 [3], spanning data structures, numerical algorithms, and web APIs.

Interrupted (50 tasks)

Runs artificially interrupted at the 50% mark to test checkpoint-based resumption.

Perturbed (50 tasks)

Runs where a single patch is modified post-hoc to test divergence detection.

All experiments use JUGE v0.9.3 on a Linux workstation (32-core AMD EPYC 7543, 256 GB RAM). Trace logs are stored to a local NVMe SSD. Cache is warm (all original runs completed successfully).

8.2 Replay overhead

Table 2: Replay overhead over original run time across strategies.

Strategy	Median overhead	95-th pct	Cache hit rate	Divergences
FULL	3.2 ms	11.8 ms	0%	0/300
INCR	1.8 ms	6.4 ms	92%	0/300
LAZY	0.9 ms	3.1 ms	92%	0/300
ADAPT	1.4 ms	5.0 ms	89%	0/300

All four strategies produce zero divergences on the standard suite, confirming Corollary 7.3 empirically. Incremental replay achieves the best balance of overhead and cache utilisation.

8.3 Interrupted replay

Table 3: Checkpoint-based resumption results (50 interrupted tasks).

Metric	Mean	Std dev
Patches recovered from checkpoint	49.8%	1.2%
Patches recovered from suffix log	49.6%	1.3%
Patches synthesised by closure	0.6%	0.4%
Total replay time	2.1 ms	0.8 ms

Only 0.6% of patches required closure synthesis, indicating that checkpoints capture nearly all descent steps and that the suffix log fills in the remainder.

8.4 Divergence detection

On the 50 perturbed tasks (one modified patch each), the divergence detector correctly identified every modified patch with zero false positives. Classification was 100% accurate: all perturbations were value-level changes and were classified as **Value** divergences. Detection latency was 0.3 ms per task.

8.5 Ablation: effect of dependency graph depth

Depth d	n	INCR overhead	Blast radius ratio
1	100	0.4 ms	8%
3	100	1.1 ms	22%
5	100	2.8 ms	41%
8	100	5.3 ms	67%

Figure 1: Incremental replay overhead vs. dependency depth for $n = 100$ patches, single changed patch.

As expected, deeper dependency graphs increase the blast radius and hence the replay overhead. The adaptive strategy alleviates this for high-depth tasks by selectively deferring deep but inexpensive patches.

9 Conclusion

We have presented Replay Gluing, a principled system for deterministically reconstructing global sections from trace logs in the JUGEo verification framework. The main contributions are:

- A formal model of the **ReplayGluer** and its interaction with four gluing strategies (§2–§3).
- An algorithm for semantic closure that handles truncated logs and checkpoint-based resumption (§4).
- A divergence detection mechanism that turns silent replay discrepancies into structured, auditable errors (§5).
- A two-level cache architecture that achieves 92% cache hit rates on the standard benchmark suite (§6).
- A proof of the Replay Determinism Theorem: for deterministic verification tasks, replay produces bit-for-bit identical global sections (§7).

Replay Gluing addresses a practical gap in formal verification infrastructure: the ability to audit, resume, and certify verification runs independently of when and where they were first executed. Together with the semantic closure and divergence detection mechanisms, it provides a complete auditability layer for the JUGEo pipeline.

Future work. An open question is whether the Replay Determinism Theorem can be extended to *probabilistic* verification tasks (randomised testing, approximate model counting) by replacing bit-for-bit equality with distributional equality. A second direction is *cross-environment replay*: recovering a global section on a machine with a different SMT solver version, using the divergence classification to route changed patches through a compatibility shim.

Acknowledgements. The author thanks the JUGEo infrastructure team for maintaining the trace-log format and the two-level cache.

References

- [1] H. Young. Grothendieck Topologies for Compositional Program Analysis. *Judgment Geometry Series*, Paper 01, 2024.
- [2] H. Young. Descent Obstructions and the Geometry of Proof Failure. *Judgment Geometry Series*, Paper 03, 2024.
- [3] H. Young. Evaluation: End-to-End Verification Benchmarks. *Judgment Geometry Series*, Paper 10, 2024.
- [4] The COQ Development Team. The COQ Proof Assistant Reference Manual, version 8.17. <https://coq.inria.fr/doc/>, 2023.
- [5] L. de Moura and S. Ullrich. The LEAN 4 Theorem Prover and Programming Language. In *CADE-28*, LNCS 12699, 2021.
- [6] K. R. M. Leino. DAFNY: An Automatic Program Verifier for Functional Correctness. In *LPAR-16*, LNCS 6355, 2010.
- [7] D. Geels, B. Chanda, A. D. Joseph, and J. Kubiawicz. Replay Debugging for Distributed Applications. In *USENIX ATC*, 2006.